



REFERENCE GUIDE

VIRTUAL CONTROL
VMWARE CLOUD FOUNDATION SOLUTIONS

VCF 9.0 Unified Command Reference

280+ CLI commands across 20 categories — ESXi, vCenter, SDDC Manager, NSX, certificate management, and API diagnostics. Single source for all VCF administration.

ESXCLI

NSXCLI

VIM-CMD

SOS

POWERCLI

CURL

VCF 9.0

VMware Cloud Foundation

VCF 9.0 Unified Command Reference

Prepared by: Virtual Control LLC **Date:** May 2026 **Document Version:** 4.1 (Option B PowerShell-sibling pass)
Classification: Internal — Lab Environment **Total Commands:** 280+ across 20 categories

PowerShell Operators: Every workstation-runnable bash block in this reference now ships with a paste-ready PowerShell sibling intended for Windows admin workstations (PS 5.1 or 7+). Translation conventions (`curl.exe` , `Invoke-RestMethod` , `here-strings` , `ConvertFrom-Json` , `cert-bypass` , `Test-NetConnection` , etc.) are documented in [../POWERSHELL-TRANSLATION-REFERENCE.md](#) . The vast majority of commands in this reference run inside the ESXi host shell, the vCenter Server appliance, the SDDC Manager appliance, or the NSX Manager CLI — those have no PowerShell equivalent and must be executed via SSH on the appliance/host. Workstation-runnable blocks (cert generation with `openssl` , REST API calls with `curl` against SDDC Manager, port and DNS testing) are accompanied by a **PowerShell equivalent:** block immediately below the bash block.

Revision History

Version	Date	Author	Changes
4.0	February 2026	Virtual Control LLC	Unified Edition merging VCF9-Ultimate-Command-Reference.md and VCF9_Combined_Command_Reference.txt into a single 280+ command reference across 20 categories.
4.1	May 2026	Virtual Control LLC	Option B PowerShell-sibling pass: every workstation-runnable bash block now has a paste-ready PowerShell equivalent. Added cross-link to ../POWERSHELL-TRANSLATION-REFERENCE.md . Each bash block annotated with its execution context (<code># bash on ESXi host</code> , <code># bash on vCenter appliance via SSH</code> , <code># bash on SDDC Manager via SSH</code> , <code># bash on workstation</code> , etc.). Appliance-only and ESXi-only blocks (<code>esxcli</code> , <code>vim-cmd</code> , <code>vmkfstools</code> , <code>vsish</code> , <code>vmkping</code> , <code>partedUtil</code> , <code>esxcfg-*</code> , <code>esxtop</code> , <code>vscsiStats</code> , <code>/etc/init.d/*</code> , <code>services.sh</code> , <code>service-control</code> , <code>vamicli</code> , <code>systemctl</code> , <code>journalctl</code> , SoS utility, <code>psql / pg_dump</code> , <code>keytool</code> , NSX appliance CLI <code>get / start / restart</code> , log file tails on appliance paths) annotated with no PowerShell sibling.

TABLE OF CONTENTS

Part I: ESXi Host Commands

- [1. System Administration](#)
- [2. Storage Management](#)
- [3. Network Commands](#)
- [4. vSAN Operations](#)
- [5. VM Management \(vim-cmd\)](#)
- [6. Service Control](#)
- [7. Performance Tools](#)

Part II: vCenter Server Commands

- [8. service-control Commands](#)
- [9. vCenter Database](#)
- [10. VAMI Tools](#)

Part III: SDDC Manager Commands

- [11. VCF Services Management](#)
- [12. SoS Utility](#)
- [13. SDDC Manager Database](#)

Part IV: NSX Manager Commands

- [14. Transport Node Operations](#)
- [15. Logical Network](#)
- [16. Distributed Firewall](#)
- [17. Cluster Operations](#)

Part V: Certificate Management

- [18. OpenSSL Commands](#)
- [19. Java Keytool Commands](#)

Part VI: API & Network Diagnostics

- [20. curl API Commands](#)
- [21. Network Testing](#)
- [22. Log Analysis](#)

Appendices

- [A. Quick Reference Cards](#)
- [B. Common Ports](#)
- [C. Log File Locations](#)
- [D. Certificate Locations](#)
- [E. Default Credentials](#)

Where commands run in this reference. This document is a multi-component command reference — each major Part is scoped to one environment:

- **PART I: ESXi HOST COMMANDS** → SSH session to an ESXi host (root). Uses `esxcli`, `vmkfstools`, `localcli`, `vim-cmd`.

- **PART II: vCenter (VCSA) COMMANDS** → SSH session to the VCSA appliance, OR vCenter API via curl/PowerCLI.
- **PART III: NSX MANAGER COMMANDS** → SSH session to the NSX Manager appliance, OR NSX-T CLI / NSX REST API.
- **PART IV: SDDC MANAGER COMMANDS** → SSH session to the SDDC Manager appliance, OR SDDC Manager REST API.
- **PART V: VCF OPERATIONS COMMANDS** → SSH session to the vcf-ops appliance, OR VCF Operations Suite API.
- **PART VI: PowerCLI** → PowerShell on a Windows workstation with VMware.PowerCLI module installed.
- **PART VII: KUBERNETES COMMANDS** → kubectl from any system with the appropriate kubeconfig.

When pasting credentials, replace placeholders like `<your-password>` with values from your vault — never paste a literal password from this document. Reference passwords are deliberately replaced with placeholders.

PART I: ESXi HOST COMMANDS

1. System Administration

1.1 Hostname Commands

```
esxcli system hostname get
```

Description: Displays the current hostname configuration including FQDN, short name, and domain name.

Syntax:

```
# bash on ESXi host (SSH)
esxcli system hostname get
```

ESXi-shell only — no PowerShell sibling.

Example Output:

```
Domain Name: lab.local
Fully Qualified Domain Name: esxi01.lab.local
Host Name: esxi01
```

Use Case: Verify hostname before certificate operations or VCF deployment.

```
esxcli system hostname set
```

Description: Sets the fully qualified domain name (FQDN) for the ESXi host.

Syntax:

```
# bash on ESXi host (SSH)
esxcli system hostname set --fqdn=<hostname.domain>
```

ESXi-shell only — no PowerShell sibling.

Parameters: | Flag | Required | Description | |-----|-----|-----| | `--fqdn` | Yes | Full hostname including domain | | `--host` | No | Short hostname only | | `--domain` | No | Domain name only |

Example:

```
# bash on ESXi host (SSH)
esxcli system hostname set --fqdn=esxi01.lab.local
```

NOTE: After changing hostname, regenerate SSL certificates with `/sbin/generate-certificates`

1.2 SSL Certificate Commands

`/sbin/generate-certificates`

Description: Regenerates the default ESXi SSL certificates using current hostname settings.

Syntax:

```
# bash on ESXi host (SSH)
/sbin/generate-certificates
```

ESXi-shell only — no PowerShell sibling.

Use Case: Run after changing FQDN to update SSL certificates.

Follow-up: Restart services with `services.sh restart`

`services.sh restart`

Description: Restarts ALL ESXi management services including hostd, vpxa.

Syntax:

```
# bash on ESXi host (SSH)
services.sh restart
```

ESXi-shell only — no PowerShell sibling.

WARNING: This causes a brief management outage. Use after certificate changes.

2. Storage Management

2.1 Device Commands

`esxcli storage core device list`

Description: Lists all storage devices with detailed information including capacity, vendor, model, and SSD status.

Syntax:

```
# bash on ESXi host (SSH)
esxcli storage core device list
```

Filter for SSD status:

```
# bash on ESXi host (SSH)
esxcli storage core device list | grep -E 'Display Name|Is SSD'
```

ESXi-shell only — no PowerShell sibling.

Use Case: Verify disk recognition and SSD detection for vSAN.

`esxcli storage core adapter rescan`

Description: Rescans storage adapters to detect newly added or changed storage devices.

Syntax:

```
# bash on ESXi host (SSH)
esxcli storage core adapter rescan --all
```

ESXi-shell only — no PowerShell sibling.

Parameters: | Flag | Required | Description | |-----|-----|-----| | `--all` | No | Rescan all adapters | | `--adapter` | No | Rescan specific adapter |

`esxcli storage filesystem list`

Description: Lists all mounted filesystems including VMFS datastores with capacity information.

Syntax:

```
# bash on ESXi host (SSH)
esxcli storage filesystem list
```

ESXi-shell only — no PowerShell sibling.

2.2 Partition Commands

`partedUtil getptbl`

Description: Displays the partition table of a disk.

Syntax:

```
# bash on ESXi host (SSH)
partedUtil getptbl /dev/disks/<device-name>
```

Example:

```
# bash on ESXi host (SSH)
partedUtil getptbl /dev/disks/naa.600508b4000156d700012
```

ESXi-shell only — no PowerShell sibling.

```
partedUtil mkLabel
```

Description: Creates a new partition table label (GPT or MBR).

Syntax:

```
# bash on ESXi host (SSH)
partedUtil mklabel /dev/disks/<device-name> gpt
```

ESXi-shell only — no PowerShell sibling.

WARNING: This destroys ALL data on the disk! Ensure correct device before running.

Use Case: Clear partitions on disks for vSAN reuse.

2.3 VMDK Tools

```
vmkfstools -D
```

Description: Displays VMDK metadata and lock information.

Syntax:

```
# bash on ESXi host (SSH)
vmkfstools -D "<path-to-vmdk>"
```

Example:

```
# bash on ESXi host (SSH)
vmkfstools -D "/vmfs/volumes/vsan:XXXX/vcenter/vcenter.vmdk"
```

ESXi-shell only — no PowerShell sibling.

Use Case: Troubleshoot VMDK locks and VM storage issues.

3. Network Commands

3.1 VMkernel Interface Commands

```
esxcli network ip interface list
```

Description: Lists all VMkernel interfaces with their configuration.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network ip interface list
```

Filter for specific interface:

```
# bash on ESXi host (SSH)
esxcli network ip interface list | grep -A10 vmk2
```

ESXi-shell only — no PowerShell sibling.

`esxcli network ip interface ipv4 get`

Description: Gets IPv4 configuration for a specific VMkernel interface.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network ip interface ipv4 get -i <vmkX>
```

Example:

```
# bash on ESXi host (SSH)
esxcli network ip interface ipv4 get -i vmk2
```

ESXi-shell only — no PowerShell sibling.

Expected Output:

Name	IPv4 Address	IPv4 Netmask	IPv4 Broadcast	Address Type
----	-----	-----	-----	-----
vmk2	192.168.12.74	255.255.255.0	192.168.12.255	STATIC

`esxcli network ip interface ipv4 set`

Description: Sets IPv4 configuration for a VMkernel interface.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network ip interface ipv4 set -i <vmkX> -I <ip> -N <netmask> -t <type>
```

Parameters: | Flag | Required | Description | |-----|-----|-----| | `-i` | Yes | Interface name (vmk0, vmk1, etc.) | | `-I` | Yes | IP address | | `-N` | Yes | Netmask | | `-t` | Yes | Address type (static/dhcp) |

Example:

```
# bash on ESXi host (SSH)
```



```
esxcli network ip interface ipv4 set -i vmk2 -I 192.168.12.74 -N 255.255.255.0 -t static
```

ESXi-shell only — no PowerShell sibling.

```
esxcfg-vmknic -l
```

Description: Lists all VMkernel interfaces with IP configuration, MTU, and enabled services.

Syntax:

```
# bash on ESXi host (SSH)
esxcfg-vmknic -l
```

ESXi-shell only — no PowerShell sibling.

Use Case: Quick view of all VMkernel adapters and their properties.

3.2 vSwitch Commands

```
esxcli network vswitch standard list
```

Description: Lists all standard vSwitches with uplinks, portgroups, and policies.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network vswitch standard list
```

Filter for uplinks:

```
# bash on ESXi host (SSH)
esxcli network vswitch standard list | grep Uplinks
```

ESXi-shell only — no PowerShell sibling.

```
esxcli network vswitch standard uplink add
```

Description: Adds a physical NIC as an uplink to a standard vSwitch.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network vswitch standard uplink add -u <vmnicX> -v <vSwitch>
```

Example:

```
# bash on ESXi host (SSH)
esxcli network vswitch standard uplink add -u vmnic3 -v vSwitch0
```

ESXi-shell only — no PowerShell sibling.

`esxcli network vswitch standard uplink remove`

Description: Removes a physical NIC from a vSwitch.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network vswitch standard uplink remove -u <vmnicX> -v <vSwitch>
```

ESXi-shell only — no PowerShell sibling.

`esxcli network vswitch standard policy failover get`

Description: Shows failover policy including active, standby, and unused adapters.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network vswitch standard policy failover get -v <vSwitch>
```

Example:

```
# bash on ESXi host (SSH)
esxcli network vswitch standard policy failover get -v vSwitch0
```

ESXi-shell only — no PowerShell sibling.

TIP: If adapters show as "Unused", they will NOT pass traffic even if assigned to the vSwitch.

`esxcli network vswitch standard policy failover set`

Description: Sets uplink failover policy to make adapters active.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network vswitch standard policy failover set -v <vSwitch> -a <vmnicX>
```

Example:

```
# bash on ESXi host (SSH)
esxcli network vswitch standard policy failover set -v vSwitch0 -a vmnic3
```

ESXi-shell only — no PowerShell sibling.

`esxcli network vswitch dvs vmware list`

Description: Lists distributed virtual switches.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network vswitch dvs vmware list
```

ESXi-shell only — no PowerShell sibling.

`esxcfg-vswitch -l`

Description: Lists all virtual switches with portgroups and uplinks in a formatted view.

Syntax:

```
# bash on ESXi host (SSH)
esxcfg-vswitch -l
```

ESXi-shell only — no PowerShell sibling.

3.3 Physical NIC Commands

`esxcli network nic list`

Description: Lists all physical NICs with link status and speed.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network nic list
```

ESXi-shell only — no PowerShell sibling.

`esxcli network nic get`

Description: Gets detailed information about a specific NIC.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network nic get -n <vmnicX>
```

ESXi-shell only — no PowerShell sibling.

`esxcli network nic stats get`

Description: Shows traffic statistics for a physical NIC.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network nic stats get -n <vmnicX>
```

Filter for packet counts:

```
# bash on ESXi host (SSH)
esxcli network nic stats get -n vmnic0 | grep -E "Packets|Bytes"
```

ESXi-shell only — no PowerShell sibling.

`esxcfg-nics -l`

Description: Lists physical NICs with driver, link state, speed, and duplex.

Syntax:

```
# bash on ESXi host (SSH)
esxcfg-nics -l
```

ESXi-shell only — no PowerShell sibling.

3.4 Routing and ARP Commands

`esxcli network ip neighbor list`

Description: Shows ARP table entries.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network ip neighbor list
```

Filter for specific subnet:

```
# bash on ESXi host (SSH)
esxcli network ip neighbor list | grep 192.168.12
```

ESXi-shell only — no PowerShell sibling.

TIP: If ARP shows "(incomplete)", the host cannot resolve the MAC address of the target.

`esxcli network ip route ipv4 list`

Description: Shows IPv4 routing table.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network ip route ipv4 list
```

ESXi-shell only — no PowerShell sibling.

`esxcli network ip connection list`

Description: Lists active network connections.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network ip connection list
```

Filter for NSX Manager communication:

```
# bash on ESXi host (SSH)
esxcli network ip connection list | grep 1234
```

ESXi-shell only — no PowerShell sibling.

3.5 Firewall Commands

`esxcli network firewall ruleset list`

Description: Lists firewall rulesets and their status.

Syntax:

```
# bash on ESXi host (SSH)
esxcli network firewall ruleset list
```

Filter for SSH:

```
# bash on ESXi host (SSH)
esxcli network firewall ruleset list | grep -i ssh
```

ESXi-shell only — no PowerShell sibling.

3.6 VMkernel Ping

`vmkping`

Description: Ping utility that uses VMkernel stack.

Syntax:

```
# bash on ESXi host (SSH)
vmkping [options] <ip-address>
```

Parameters: | Flag | Description | |-----|-----| | `-I` | Specify source interface | | `-d` | Don't fragment | | `-s` | Packet size | | `-c` | Count of pings |

Examples:

```
# bash on ESXi host (SSH)
# Basic ping
vmkping 192.168.12.75

# Ping from specific interface
vmkping -I vmk2 192.168.12.75

# MTU test (1600 bytes total)
vmkping -d -s 1572 192.168.12.75

# Ping with count
vmkping -c 10 192.168.12.75
```

ESXi-shell only — no PowerShell sibling. (`vmkping` runs from the VMkernel stack and has no host-OS equivalent; from a workstation use `Test-Connection` or `ping` , but those traverse the management network only.)

4. vSAN Operations

4.1 Cluster Commands

`esxcli vsan cluster get`

Description: Displays current vSAN cluster information and membership status.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan cluster get
```

ESXi-shell only — no PowerShell sibling.

Key Fields to Check: - Sub-Cluster Member Count - Should match total host count - Local Node State - Should be MASTER, BACKUP, or AGENT - Local Node Health State - Should be HEALTHY

`esxcli vsan cluster leave`

Description: Forces the ESXi host to leave its current vSAN cluster.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan cluster leave
```

ESXi-shell only — no PowerShell sibling.

WARNING: Use with caution. This removes the host from the vSAN cluster.

`esxcli vsan cluster unicastagent list`

Description: Lists unicast agents to see all cluster members.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan cluster unicastagent list
```

ESXi-shell only — no PowerShell sibling.

4.2 Storage Commands

```
esxcli vsan storage automode set
```

Description: Enables or disables vSAN automatic disk claiming.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan storage automode set --enabled=<true|false>
```

Examples:

```
# bash on ESXi host (SSH)
# Disable auto-claim for manual disk selection
esxcli vsan storage automode set --enabled=false

# Enable auto-claim (default)
esxcli vsan storage automode set --enabled=true
```

ESXi-shell only — no PowerShell sibling.

```
esxcli vsan storage list
```

Description: Lists vSAN storage devices and disk groups.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan storage list
```

ESXi-shell only — no PowerShell sibling.

```
esxcli vsan storage add
```

Description: Adds storage devices to vSAN.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan storage add -s <cache-device> -d <capacity-device>
```

ESXi-shell only — no PowerShell sibling.

Parameters: | Flag | Required | Description | |-----|-----|-----| | -s | Yes | Cache tier device | | -d | Yes | Capacity tier device |

`esxcli vsan storage remove`

Description: Removes a device from vSAN.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan storage remove -s <device>
```

ESXi-shell only — no PowerShell sibling.

4.3 Health Commands

`esxcli vsan health cluster list`

Description: Lists vSAN health checks and their status.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan health cluster list
```

ESXi-shell only — no PowerShell sibling.

`esxcli vsan health cluster get`

Description: Gets specific health test results.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan health cluster get -t "vSAN Health"
```

ESXi-shell only — no PowerShell sibling.

4.4 Network Commands

`esxcli vsan network list`

Description: Lists vSAN network adapters.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan network list
```

ESXi-shell only — no PowerShell sibling.

```
esxcli vsan network ip add
```

Description: Adds a VMkernel interface to vSAN.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan network ip add -i <vmkX>
```

Example:

```
# bash on ESXi host (SSH)
esxcli vsan network ip add -i vmk1
```

ESXi-shell only — no PowerShell sibling.

4.5 Debug Commands

```
esxcli vsan debug resync summary
```

Description: Shows vSAN resync status and progress.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan debug resync summary
```

ESXi-shell only — no PowerShell sibling.

```
esxcli vsan debug object list
```

Description: Lists vSAN objects for debugging.

Syntax:

```
# bash on ESXi host (SSH)
esxcli vsan debug object list
```

ESXi-shell only — no PowerShell sibling.

5. VM Management (vim-cmd)

5.1 VM Listing

```
vim-cmd vmsvc/getallvms
```

Description: Lists all registered VMs with their VMIDs and configuration file paths.

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd vmsvc/getallvms
```

ESXi-shell only — no PowerShell sibling. (For workstation-side VM inventory, use PowerCLI: `Get-VM` against vCenter.)

5.2 Power Operations

```
vim-cmd vmsvc/power.getstate
```

Description: Gets the power state of a specific VM.

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd vmsvc/power.getstate <vmid>
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative against vCenter: `(Get-VM <name>).PowerState`.)

```
vim-cmd vmsvc/power.on
```

Description: Powers on a virtual machine.

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd vmsvc/power.on <vmid>
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative against vCenter: `Start-VM -VM <name>`.)

```
vim-cmd vmsvc/power.off
```

Description: Powers off a virtual machine (hard power off).

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd vmsvc/power.off <vmid>
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative against vCenter: `Stop-VM -VM <name> -Kill -Confirm:$false`.)

WARNING: This is a hard power off. Use `power.shutdown` for graceful shutdown.

```
vim-cmd vmsvc/power.reset
```

Description: Resets (hard reboot) a virtual machine.

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd vmsvc/power.reset <vmid>
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative against vCenter: `Restart-VM -VM <name> -Confirm:$false .`)

```
vim-cmd vmsvc/power.shutdown
```

Description: Gracefully shuts down a VM (requires VMware Tools).

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd vmsvc/power.shutdown <vmid>
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative against vCenter: `Shutdown-VMGuest -VM <name> -Confirm:$false .`)

5.3 VM Registration

```
vim-cmd solo/registervm
```

Description: Registers a VM from its VMX file.

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd solo/registervm "<path-to-vmx>"
```

Example:

```
# bash on ESXi host (SSH)
vim-cmd solo/registervm "/vmfs/volumes/vsan:XXXX/vcenter/vcenter.vmx"
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative when vCenter is alive: `New-VM -VMFilePath '[datastore] path/file.vmx' .`)

Use Case: Recover VMs from datastore after vCenter failure.

```
vim-cmd vmsvc/unregister
```

Description: Unregisters a VM (does not delete files).

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd vmsvc/unregister <vmid>
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative against vCenter: `Remove-VM -VM <name> -DeletePermanently:$false -Confirm:$false .`)

5.4 VM Configuration

```
vim-cmd vmsvc/device.getdevices
```

Description: Lists all devices attached to a VM.

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd vmsvc/device.getdevices <vmid>
```

Filter for disk files:

```
# bash on ESXi host (SSH)
vim-cmd vmsvc/device.getdevices <vmid> | grep -i "fileName\|label"
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative: `Get-VM <name> | Get-HardDisk` and `Get-NetworkAdapter`.)

5.5 Host Maintenance

```
vim-cmd hostsvc/maintenance_mode_enter
```

Description: Puts the host into maintenance mode.

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd hostsvc/maintenance_mode_enter
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative against vCenter: `Set-VMHost -VMHost <host> -State Maintenance`.)

```
vim-cmd hostsvc/maintenance_mode_exit
```

Description: Takes the host out of maintenance mode.

Syntax:

```
# bash on ESXi host (SSH)
vim-cmd hostsvc/maintenance_mode_exit
```

ESXi-shell only — no PowerShell sibling. (PowerCLI alternative against vCenter: `Set-VMHost -VMHost <host> -State Connected`.)

6. ESXi Service Control

6.1 Host Daemon (hostd)

`/etc/init.d/hostd`

Description: Controls the host daemon (hostd) service.

Syntax:

```
# bash on ESXi host (SSH)
/etc/init.d/hostd <action>
```

Actions: | Action | Description | |-----|-----| | `start` | Start the service | | `stop` | Stop the service | | `restart` | Restart the service | | `status` | Check service status |

Example:

```
# bash on ESXi host (SSH)
/etc/init.d/hostd restart
```

ESXi-shell only — no PowerShell sibling.

6.2 vCenter Agent (vpxa)

`/etc/init.d/vpxa`

Description: Controls the vCenter agent (vpxa) service.

Syntax:

```
# bash on ESXi host (SSH)
/etc/init.d/vpxa restart
```

ESXi-shell only — no PowerShell sibling.

Use Case: Restart when host disconnects from vCenter.

6.3 SSH Service

`/etc/init.d/SSH`

Description: Controls the SSH service (TSM-SSH).

Syntax:

```
# bash on ESXi host (SSH)
/etc/init.d/SSH status
/etc/init.d/SSH start
/etc/init.d/SSH stop
```

ESXi-shell only — no PowerShell sibling.

6.4 NSX Services on ESXi

`/etc/init.d/nsx-proxy`

Description: Controls the NSX proxy agent on ESXi.

Syntax:

```
# bash on ESXi host (SSH)
/etc/init.d/nsx-proxy status
/etc/init.d/nsx-proxy restart
```

ESXi-shell only — no PowerShell sibling.

`/etc/init.d/nsx-opsagent`

Description: Controls the NSX operations agent.

Syntax:

```
# bash on ESXi host (SSH)
/etc/init.d/nsx-opsagent status
```

ESXi-shell only — no PowerShell sibling.

`/etc/init.d/nsx-datapath`

Description: Controls the NSX datapath service for distributed firewall.

Syntax:

```
# bash on ESXi host (SSH)
/etc/init.d/nsx-datapath status
```

ESXi-shell only — no PowerShell sibling.

7. Performance Tools

7.1 esxtop

`esxtop` *(Interactive)*

Description: Interactive real-time performance monitoring tool.

Syntax:

```
# bash on ESXi host (SSH)
esxtop
```

ESXi-shell only — no PowerShell sibling.

Interactive Views: | Key | View | |-----|-----| | **c** | CPU | | **m** | Memory | | **n** | Network | | **d** | Disk/Storage | | **v** | VM view | | **u** | Disk device |

`esxtop` *(Batch Mode)*

Description: Captures performance data to CSV file.

Syntax:

```
# bash on ESXi host (SSH)
esxtop -b -d <interval> -n <iterations> > <output-file>
```

Example:

```
# bash on ESXi host (SSH)
esxtop -b -d 5 -n 10 > /tmp/esxtop.csv
```

ESXi-shell only — no PowerShell sibling. (Workstation alternative: `Get-Stat -Entity (Get-VMHost) -Stat cpu.usage.average,mem.usage.average -Realtime` via PowerCLI.)

Parameters: | Flag | Description | |-----|-----| | **-b** | Batch mode | | **-d** | Delay between samples (seconds) | | **-n** | Number of iterations |

7.2 vscsiStats

`vscsiStats -l`

Description: Lists VMs available for storage statistics collection.

Syntax:

```
# bash on ESXi host (SSH)
vscsiStats -l
```

ESXi-shell only — no PowerShell sibling.

`vscsiStats -s`

Description: Starts collecting storage statistics for a VM.

Syntax:

```
# bash on ESXi host (SSH)
vscsiStats -s -w <world-id>
```

ESXi-shell only — no PowerShell sibling.

`vscsiStats -p`

Description: Prints storage statistics for a VM.

Syntax:

```
# bash on ESXi host (SSH)
vscsiStats -p all -w <world-id>
```

ESXi-shell only — no PowerShell sibling.

7.3 vsish

`vsish`

Description: VMkernel System Information Shell for advanced diagnostics.

Syntax:

```
# bash on ESXi host (SSH)
vsish -e <command> <path>
```

Examples:

```
# bash on ESXi host (SSH)
# List vsish nodes
vsish -e ls /

# Get memory statistics
vsish -e get /memory/comprehensive

# Get network portset info
vsish -e get /net/portsets/
```

ESXi-shell only — no PowerShell sibling.

PART II: VCENTER SERVER COMMANDS

8. service-control Commands

8.1 Status Commands

`service-control --status --all`

Description: Shows status of all vCenter services.

Syntax:

```
# bash on vCenter appliance via SSH
service-control --status --all
```

vCenter-appliance only — no PowerShell sibling.

Key Services: | Service | Description | |-----|-----| | vpxd | vCenter Server daemon | | vsphere-client | vSphere Web Client | | vmware-vpostgres | PostgreSQL database | | vmware-sps | Profile-Driven Storage | | vmcad | Certificate Authority | | vmdird | Directory Service | | vmafdd | Authentication Framework |

`service-control --status`

Description: Shows status of a specific service.

Syntax:

```
# bash on vCenter appliance via SSH
service-control --status <service-name>
```

vCenter-appliance only — no PowerShell sibling.

8.2 Start/Stop Commands

`service-control --start --all`

Description: Starts all vCenter services.

Syntax:

```
# bash on vCenter appliance via SSH
service-control --start --all
```

vCenter-appliance only — no PowerShell sibling.

`service-control --stop --all`

Description: Stops all vCenter services.

Syntax:

```
# bash on vCenter appliance via SSH
service-control --stop --all
```

vCenter-appliance only — no PowerShell sibling.

8.3 Restart Commands

`service-control --restart`

Description: Restarts a specific service.

Syntax:

```
# bash on vCenter appliance via SSH
service-control --restart <service-name>
```

Examples:

```
# bash on vCenter appliance via SSH
# Restart vCenter daemon
service-control --restart vpxd

# Restart vSphere Client
service-control --restart vsphere-client

# Restart database
service-control --restart vmware-vpostgres
```

vCenter-appliance only — no PowerShell sibling.

```
service-control --restart --all
```

Description: Restarts all vCenter services.

Syntax:

```
# bash on vCenter appliance via SSH
service-control --restart --all
```

vCenter-appliance only — no PowerShell sibling.

WARNING: This causes a brief vCenter outage.

9. vCenter Database

9.1 PostgreSQL Access

Connect to vCenter Database

```
# bash on vCenter appliance via SSH
/opt/vmware/vpostgres/current/bin/psql -U postgres
```

vCenter-appliance only — no PowerShell sibling.

Test Database Connection

```
# bash on vCenter appliance via SSH
/opt/vmware/vpostgres/current/bin/psql -U postgres -c "SELECT count(*) FROM pg_stat_activity;"
```

vCenter-appliance only — no PowerShell sibling.

10. VAMI Tools

10.1 Version and Updates

`vamcli version`

Description: Shows appliance version information.

Syntax:

```
# bash on vCenter appliance via SSH
vamcli version --appliance
```

vCenter-appliance only — no PowerShell sibling.

`vamcli update`

Description: Checks for available updates.

Syntax:

```
# bash on vCenter appliance via SSH
vamcli update --check
```

vCenter-appliance only — no PowerShell sibling.

VAMI Login

```
# bash on vCenter appliance via SSH
/opt/vmware/share/vami/vami_login
```

vCenter-appliance only — no PowerShell sibling.

PART III: SDDC MANAGER COMMANDS

11. VCF Services Management

11.1 Bundle Service Control

`systemctl status vcf-services`

Description: Shows status of all VCF services bundle.

Syntax:

```
# bash on SDDC Manager via SSH
systemctl status vcf-services
```

SDDC Manager appliance only — no PowerShell sibling.

`systemctl restart vcf-services`

Description: Restarts all VCF services.

Syntax:

```
# bash on SDDC Manager via SSH
systemctl restart vcf-services
```

SDDC Manager appliance only — no PowerShell sibling.

`systemctl stop vcf-services`

Description: Stops all VCF services.

Syntax:

```
# bash on SDDC Manager via SSH
systemctl stop vcf-services
```

SDDC Manager appliance only — no PowerShell sibling.

`systemctl start vcf-services`

Description: Starts all VCF services.

Syntax:

```
# bash on SDDC Manager via SSH
systemctl start vcf-services
```

SDDC Manager appliance only — no PowerShell sibling.

11.2 Individual Service Control

Domain Manager

```
# bash on SDDC Manager via SSH
systemctl status domainmanager
systemctl restart domainmanager
```

SDDC Manager appliance only — no PowerShell sibling.

Lifecycle Manager

```
# bash on SDDC Manager via SSH
systemctl status lcm
```

```
systemctl restart lcm
systemctl restart lcm.service
```

SDDC Manager appliance only — no PowerShell sibling.

Operations Manager

```
# bash on SDDC Manager via SSH
systemctl status operationsmanager
systemctl restart operationsmanager
```

SDDC Manager appliance only — no PowerShell sibling.

NGINX

```
# bash on SDDC Manager via SSH
systemctl status nginx
systemctl restart nginx
```

SDDC Manager appliance only — no PowerShell sibling.

PostgreSQL

```
# bash on SDDC Manager via SSH
systemctl status postgresql
systemctl restart postgresql
```

SDDC Manager appliance only — no PowerShell sibling.

SDDC Manager UI

```
# bash on SDDC Manager via SSH
systemctl restart sddc-manager-ui-app.service
```

SDDC Manager appliance only — no PowerShell sibling.

11.3 Service Discovery

`systemctl list-units`

Description: Lists systemd units with optional filtering.

Syntax:

```
# bash on SDDC Manager via SSH
systemctl list-units --type=service | grep vcf
```

SDDC Manager appliance only — no PowerShell sibling.

12. SoS Utility

Located at `/opt/vmware/sddc-support/sos`

12.1 Log Collection

`sos --log-bundle`

Description: Collects comprehensive log bundle for VMware support.

Syntax:

```
# bash on SDDC Manager via SSH
/opt/vmware/sddc-support/sos --log-bundle
```

SDDC Manager appliance only — no PowerShell sibling.

`sos --domain-name`

Description: Collects logs for a specific workload domain.

Syntax:

```
# bash on SDDC Manager via SSH
/opt/vmware/sddc-support/sos --domain-name <domain>
```

SDDC Manager appliance only — no PowerShell sibling.

12.2 Health and Inventory

`sos --health-check`

Description: Runs health check on SDDC Manager and components.

Syntax:

```
# bash on SDDC Manager via SSH
/opt/vmware/sddc-support/sos --health-check
```

SDDC Manager appliance only — no PowerShell sibling.

`sos --get-inventory`

Description: Gets inventory of all VCF components.

Syntax:

```
# bash on SDDC Manager via SSH
/opt/vmware/sddc-support/sos --get-inventory
```

SDDC Manager appliance only — no PowerShell sibling.

12.3 Maintenance

`sos --cleanup-logs`

Description: Cleans up old log bundles to free disk space.

Syntax:

```
# bash on SDDC Manager via SSH
/opt/vmware/sddc-support/sos --cleanup-logs
```

SDDC Manager appliance only — no PowerShell sibling.

`sos --get-passwords`

Description: Retrieves current passwords (requires authentication).

Syntax:

```
# bash on SDDC Manager via SSH
/opt/vmware/sddc-support/sos --get-passwords
```

SDDC Manager appliance only — no PowerShell sibling.

`sos --backup-config`

Description: Backs up SDDC Manager configuration.

Syntax:

```
# bash on SDDC Manager via SSH
/opt/vmware/sddc-support/sos --backup-config
```

SDDC Manager appliance only — no PowerShell sibling.

13. SDDC Manager Database

13.1 PostgreSQL Access

Connect to SDDC Manager Database

```
# bash on SDDC Manager via SSH
psql -h localhost -U postgres -d sddc
```

SDDC Manager appliance only — no PowerShell sibling.

13.2 psql Internal Commands

Command	Description
\dt	List all tables
\l	List databases
\q	Exit psql
\?	Help

13.3 Backup and Restore

Backup Database

```
# bash on SDDC Manager via SSH
pg_dump -U postgres sddc > /tmp/sddc_backup.sql
```

SDDC Manager appliance only — no PowerShell sibling.

Restore Database

```
# bash on SDDC Manager via SSH
psql -U postgres sddc < /tmp/sddc_backup.sql
```

SDDC Manager appliance only — no PowerShell sibling.

13.4 Health Check

Test Connection

```
# bash on SDDC Manager via SSH
sudo -u postgres psql -c "SELECT 1;"
```

SDDC Manager appliance only — no PowerShell sibling.

PART IV: NSX MANAGER COMMANDS

Context for all NSX commands in Sections 14-17: These are NSX Manager CLI commands — SSH into the NSX Manager VM as the `admin` user (NOT root), then run them at the `nsxmanager>` prompt. They are NOT bash; they have no PowerShell equivalent. To query the same data from a Windows workstation, use the NSX Manager REST API via `Invoke-Res` `tMethod` against `https://<nsx-manager>/api/v1/...` or `/policy/api/v1/...`.

14. Transport Node Operations

14.1 Node Listing

get transport-nodes

Description: Lists all transport nodes.

Syntax:

```
get transport-nodes
```

get transport-node status

Description: Gets detailed status of a specific transport node.

Syntax:

```
get transport-node <uuid> status
```

15. Logical Network

15.1 Switches and Routers

get logical-switches

Description: Lists all logical switches (segments).

Syntax:

```
get logical-switches
```

get logical-routers

Description: Lists all logical routers.

Syntax:

```
get logical-routers
```

15.2 VTEP/TEP Operations

get vtep

Description: Shows VTEP information.

Syntax:

```
get vtep
```

get vtep-table

Description: Displays VTEP table entries.

Syntax:

```
get vtep-table
```

get interfaces

Description: Lists all interfaces.

Syntax:

```
get interfaces
```

15.3 Traceflow

start traceflow

Description: Initiates a traceflow for network debugging.

Syntax:

```
start traceflow --src-port <port-id> --dst-ip <ip>
```

get traceflow

Description: Gets results of a traceflow operation.

Syntax:

```
get traceflow <traceflow-id>
```

16. Distributed Firewall

16.1 Rule Management

get firewall rules

Description: Lists all DFW rules.

Syntax:

```
get firewall rules
```

`get firewall status`

Description: Checks DFW status.

Syntax:

```
get firewall status
```

`get firewall rule`

Description: Gets specific rule details.

Syntax:

```
get firewall rule <rule-id>
```

17. NSX Cluster Operations

17.1 Cluster Status

`get cluster status`

Description: Checks NSX controller cluster status.

Syntax:

```
get cluster status
```

`get cluster nodes`

Description: Lists cluster nodes.

Syntax:

```
get cluster nodes
```

`get managers`

Description: Lists NSX Manager nodes.

Syntax:

```
get managers
```

PART V: CERTIFICATE MANAGEMENT

18. OpenSSL Commands

18.1 Certificate Generation

Self-Signed Certificate

Description: Generates a self-signed certificate and private key.

Syntax:

```
# bash on workstation (Git Bash / WSL / Linux / macOS)
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt -days 365 -nodes -subj
'/CN=hostname'
```

PowerShell equivalent:

```
# PowerShell on workstation -- requires openssl.exe in PATH (Git for Windows or standalone)
openssl req -x509 -newkey rsa:2048 `
  -keyout server.key `
  -out server.crt `
  -days 365 -nodes `
  -subj "/CN=hostname"
```

Parameters: | Flag | Description | |-----|-----| | -x509 | Generate self-signed certificate | | -newkey rsa:2048 | Create new 2048-bit RSA key | | -keyout | Output file for private key | | -out | Output file for certificate | | -days 365 | Validity period | | -nodes | No passphrase | | -subj | Subject line (requires leading slash) |

Extended Example with SANs:

```
# bash on workstation
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt -days 365 -nodes \
  -subj "/CN=192.168.1.52/O=VCF-Depot/C=US" \
  -addext "subjectAltName=IP:192.168.1.52,DNS:localhost" \
  -addext "keyUsage=digitalSignature,keyEncipherment" \
  -addext "extendedKeyUsage=serverAuth"
```

PowerShell equivalent:

```
# PowerShell on workstation -- requires openssl.exe in PATH
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt -days 365 -nodes `
  -subj "/CN=192.168.1.52/O=VCF-Depot/C=US" `
  -addext "subjectAltName=IP:192.168.1.52,DNS:localhost" `
  -addext "keyUsage=digitalSignature,keyEncipherment" `
  -addext "extendedKeyUsage=serverAuth"
```

18.2 Certificate Viewing

View Certificate Details

Description: Displays full certificate details.

Syntax:

```
# bash on workstation OR appliance (file path determines location)
openssl x509 -in <certificate> -text -noout
```

PowerShell equivalent:

```
# PowerShell on workstation -- requires openssl.exe in PATH
openssl x509 -in <certificate> -text -noout
```

Example (ESXi-side certificate):

```
# bash on ESXi host (SSH) -- /etc/vmware/ssl/rui.crt is an appliance path
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout
```

ESXi-shell only — no PowerShell sibling. To inspect an ESXi cert from a workstation, copy it off first (`scp root@esxi:/etc/vmware/ssl/rui.crt .`) then run the workstation form above.

View Subject Alternative Names

Description: Shows only the SAN field from a certificate.

Syntax:

```
# bash on workstation
openssl x509 -in <certificate> -text -noout | grep -A1 'Subject Alternative Name'
```

PowerShell equivalent:

```
# PowerShell on workstation -- requires openssl.exe in PATH
(openssl x509 -in <certificate> -text -noout) -match 'Subject Alternative Name' -or
((openssl x509 -in <certificate> -text -noout) | Select-String -Pattern 'Subject Alternative Name' -
Context 0,1)
# Or simpler: pipe to Select-String with context
openssl x509 -in <certificate> -text -noout | Select-String -Pattern 'Subject Alternative Name' -Context
0,1
```

View Certificate Dates

Description: Shows validity dates.

Syntax:

```
# bash on workstation
openssl x509 -in cert.crt -noout -dates
```

PowerShell equivalent:

```
# PowerShell on workstation -- requires openssl.exe in PATH
openssl x509 -in cert.crt -noout -dates

# Or pure-PowerShell (no openssl needed):
$cert = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2 'cert.crt'
"notBefore=$($cert.NotBefore.ToString('MMM d HH:mm:ss yyyy GMT'))"
"notAfter=$($cert.NotAfter.ToString('MMM d HH:mm:ss yyyy GMT'))"
```

View Expiration Only

```
# bash on workstation
openssl x509 -in cert.crt -noout -enddate
```

PowerShell equivalent:

```
# PowerShell on workstation -- pure PowerShell, no openssl needed
$cert = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2 'cert.crt'
"notAfter=$($cert.NotAfter.ToString('MMM d HH:mm:ss yyyy GMT'))"
```

18.3 Certificate Verification

Verify Against CA

Description: Verifies a certificate against a CA certificate.

Syntax:

```
# bash on workstation
openssl verify -CAfile ca.crt server.crt
```

PowerShell equivalent:

```
# PowerShell on workstation -- requires openssl.exe in PATH
openssl verify -CAfile ca.crt server.crt
```

View Remote Certificate

Description: Connects to a server and displays its certificate chain.

Syntax:

```
# bash on workstation
openssl s_client -connect <host>:443 -showcerts
```

PowerShell equivalent:

```
# PowerShell on workstation -- requires openssl.exe in PATH (preferred for full chain)
openssl s_client -connect <host>:443 -showcerts

# Or pure-PowerShell (single-cert view, no chain):
$tcp = [System.Net.Sockets.TcpClient]::new('<host>', 443)
$ssl = [System.Net.Security.SslStream]::new($tcp.GetStream(), $false, { $true })
```

```
$ssl.AuthenticateAsClient('<host>')  
[System.Security.Cryptography.X509Certificates.X509Certificate2]$ssl.RemoteCertificate | Format-List  
Subject, Issuer, NotBefore, NotAfter, Thumbprint  
$ssl.Close(); $tcp.Close()
```

Example:

```
# bash on workstation  
openssl s_client -connect vcenter.lab.local:443 -showcerts
```

PowerShell equivalent:

```
# PowerShell on workstation  
openssl s_client -connect vcenter.lab.local:443 -showcerts
```

19. Java Keytool Commands

19.1 Import Certificate

`keytool -import`

Description: Imports a certificate into a Java truststore.

Syntax:

```
# bash on SDDC Manager / vCenter / NSX appliance via SSH  
keytool -import -trustcacerts -alias <name> -file <cert> -keystore <cacerts> -storepass changeit -noprompt
```

Example:

```
# bash on SDDC Manager / vCenter / NSX appliance via SSH  
keytool -import -trustcacerts -alias vcf-depot \  
-file /tmp/depot.crt \  
-keystore /usr/lib/jvm/openjdk-java17-headless.x86_64/lib/security/cacerts \  
-storepass changeit -noprompt
```

Appliance-only — no PowerShell sibling. The keystore path (`/usr/lib/jvm/...`) is on the VCF appliance JVM. Always SSH into the appliance to modify its truststore.

NOTE: Default Java Keystore password is `changeit`

19.2 List Certificates

`keytool -list`

Description: Lists all certificates in a keystore.

Syntax:

```
# bash on SDDC Manager / vCenter / NSX appliance via SSH
keytool -list -keystore <cacerts> -storepass changeit
```

Verbose listing:

```
# bash on SDDC Manager / vCenter / NSX appliance via SSH
keytool -list -v -keystore <cacerts> -storepass changeit
```

Appliance-only — no PowerShell sibling. Run on the appliance whose truststore you are inspecting.

19.3 Delete Certificate

`keytool -delete`

Description: Removes a certificate from a keystore.

Syntax:

```
# bash on SDDC Manager / vCenter / NSX appliance via SSH
keytool -delete -alias <name> -keystore <cacerts> -storepass changeit
```

Appliance-only — no PowerShell sibling.

19.4 VCF Trust Store

List VCF Trust Store

```
# bash on SDDC Manager via SSH
keytool -list -v -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store -storepass <password>
```

SDDC Manager appliance only — no PowerShell sibling. The trust store path is on the SDDC Manager appliance.

PART VI: API & NETWORK DIAGNOSTICS

20. curl API Commands

PowerShell 5.1 cert-bypass note: The PowerShell siblings below assume you have already added the standard self-signed-cert trust shim once per session. If you see "Could not establish trust relationship" errors, run this first:


```
```powershell
```

# PowerShell on workstation -- once per session for PS 5.1

---

```
add-type @" using System.Net; using System.Security.Cryptography.X509Certificates; public class TrustAllCerts :
ICertificatePolicy { public bool CheckValidationResult(ServicePoint sp, X509Certificate cert, WebRequest req, int problem) {
return true; } } "@ [System.Net.ServicePointManager]::CertificatePolicy = New-Object TrustAllCerts
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12 ```
```

On PowerShell 7+, use `-SkipCertificateCheck` on each `Invoke-RestMethod` call instead.

## 20.1 SDDC Manager API

### Get Authentication Token

**Description:** Authenticates and retrieves a bearer token.

**Syntax:**

```
bash on workstation
curl -k -X POST https://<sddc-manager>/v1/tokens \
-H "Content-Type: application/json" \
-d '{"username":"admin@local","password":"<password>"}'
```

**PowerShell equivalent:**

```
PowerShell on workstation
$body = @{ username = 'admin@local'; password = '<password>' } | ConvertTo-Json
$resp = Invoke-RestMethod -Method POST `
 -Uri "https://<sddc-manager>/v1/tokens" `
 -Body $body -ContentType 'application/json'
$token = $resp.accessToken
$headers = @{ Authorization = "Bearer $token" }
(PS 7+: append -SkipCertificateCheck. PS 5.1: ensure cert-bypass shim above is loaded.)
```

---

### Get System Information

```
bash on workstation
curl -k -X GET https://<sddc-manager>/v1/system \
-H "Authorization: Bearer <token>"
```

**PowerShell equivalent:**

```
PowerShell on workstation -- $headers from token call above
Invoke-RestMethod -Method GET `
 -Uri "https://<sddc-manager>/v1/system" `
 -Headers $headers
```

---

### List Hosts

```
bash on workstation
curl -k -X GET https://<sddc-manager>/v1/hosts \
-H "Authorization: Bearer <token>"
```

### PowerShell equivalent:

```
PowerShell on workstation -- $headers from token call above
Invoke-RestMethod -Method GET `
-Uri "https://<sddc-manager>/v1/hosts" `
-Headers $headers | Select-Object -ExpandProperty elements
```

---

## List Domains

```
bash on workstation
curl -k -X GET https://<sddc-manager>/v1/domains \
-H "Authorization: Bearer <token>"
```

### PowerShell equivalent:

```
PowerShell on workstation -- $headers from token call above
Invoke-RestMethod -Method GET `
-Uri "https://<sddc-manager>/v1/domains" `
-Headers $headers | Select-Object -ExpandProperty elements
```

---

## List Tasks

```
bash on workstation
curl -k -X GET https://<sddc-manager>/v1/tasks \
-H "Authorization: Bearer <token>"
```

### PowerShell equivalent:

```
PowerShell on workstation -- $headers from token call above
Invoke-RestMethod -Method GET `
-Uri "https://<sddc-manager>/v1/tasks" `
-Headers $headers | Select-Object -ExpandProperty elements
```

---

## Cancel Stuck Task

```
bash on workstation
curl -k -X PATCH https://<sddc-manager>/v1/tasks/<task-id> \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"status":"CANCELLED"}'
```

### PowerShell equivalent:

```
PowerShell on workstation -- $headers from token call above
$cancelBody = @{ status = 'CANCELLED' } | ConvertTo-Json
Invoke-RestMethod -Method PATCH `
-Uri "https://<sddc-manager>/v1/tasks/<task-id>" `
```

```
-Headers $headers `
-Body $cancelBody -ContentType 'application/json'
```

## 20.2 General curl Commands

### Download with Authentication

```
bash on workstation
curl -k -u <user>:<pass> -O <URL>
```

#### PowerShell equivalent:

```
PowerShell on workstation
$cred = [pscredential]::new('<user>', (ConvertTo-SecureString '<pass>' -AsPlainText -Force))
$fileName = Split-Path -Leaf '<URL>'
Invoke-WebRequest -Uri '<URL>' -Credential $cred -OutFile $fileName
(PS 7+: append -SkipCertificateCheck for self-signed. PS 5.1: load cert-bypass shim above.)
```

### Fetch HTTP Headers Only

```
bash on workstation
curl -I <URL>
```

#### PowerShell equivalent:

```
PowerShell on workstation
(Invoke-WebRequest -Uri '<URL>' -Method Head).Headers
```

### Download and Save

```
bash on workstation
curl -k -O <URL>
```

#### PowerShell equivalent:

```
PowerShell on workstation
$fileName = Split-Path -Leaf '<URL>'
Invoke-WebRequest -Uri '<URL>' -OutFile $fileName
(PS 7+: append -SkipCertificateCheck. PS 5.1: load cert-bypass shim above.)
```

## 20.3 API Status Codes

Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized

Code	Meaning
403	Forbidden
404	Not Found
500	Internal Server Error

## 21. Network Testing

### 21.1 Port Testing

**nc** (*netcat*)

**Description:** Tests TCP port connectivity.

**Syntax:**

```
bash on workstation OR appliance
nc -zv <ip> <port>
```

**PowerShell equivalent:**

```
PowerShell on workstation -- native cmdlet, no extra tooling
Test-NetConnection -ComputerName <ip> -Port <port>
```

**Parameters:** | Flag | Description | |-----|-----| | -z | Scan without sending data | | -v | Verbose output | | -w | Timeout in seconds |

**Examples:**

```
bash on workstation OR appliance
Test SSH port
nc -zv 192.168.1.74 22

Test HTTPS with timeout
nc -zv 192.168.1.74 443 -w 5

Test NFS port
nc -zv 192.168.1.52 2049

Test portmapper
nc -zv 192.168.1.52 111
```

**PowerShell equivalent:**

```
PowerShell on workstation
Test SSH port
Test-NetConnection -ComputerName 192.168.1.74 -Port 22

Test HTTPS (Test-NetConnection has no native timeout flag; use a job for timeout control)
Test-NetConnection -ComputerName 192.168.1.74 -Port 443

Test NFS port
Test-NetConnection -ComputerName 192.168.1.52 -Port 2049
```

```
Test portmapper
Test-NetConnection -ComputerName 192.168.1.52 -Port 111

Concise true/false form:
(Test-NetConnection -ComputerName 192.168.1.74 -Port 443 -WarningAction SilentlyContinue).TcpTestSucceeded
```

---

## 21.2 DNS Testing

### *nslookup*

**Description:** Queries DNS servers to resolve hostnames.

**Syntax:**

```
bash on workstation OR appliance
nslookup <hostname>
```

**PowerShell equivalent:**

```
PowerShell on workstation -- nslookup.exe is built into Windows
nslookup <hostname>

Or native PowerShell cmdlet:
Resolve-DnsName -Name <hostname>
```

**Example:**

```
bash on workstation
nslookup vcenter.lab.local
```

**PowerShell equivalent:**

```
PowerShell on workstation
Resolve-DnsName -Name vcenter.lab.local
```

---

### *View DNS Configuration*

```
bash on appliance (Linux) or workstation Git-Bash style
cat /etc/resolv.conf
```

**PowerShell equivalent:**

```
PowerShell on workstation -- Windows stores DNS settings per NIC
Get-DnsClientServerAddress | Where-Object {$_.AddressFamily -eq 2}
Or for full config:
Get-DnsClient
```

---

## 21.3 NFS Testing

### *Show NFS Exports*

```
bash on appliance OR workstation (with NFS client tools)
showmount -e <nfs-server>
```

#### PowerShell equivalent:

```
PowerShell on workstation -- requires Windows NFS client feature installed
Enable once: Enable-WindowsOptionalFeature -Online -FeatureName ServicesForNFS-ClientOnly,ClientForNFS-Infrastructure -All
showmount -e <nfs-server>
Note: showmount.exe ships with the Windows "Services for NFS / Client for NFS" optional feature.
```

#### List NFS Mounts

```
bash on appliance
mount | grep nfs
```

#### PowerShell equivalent:

```
PowerShell on workstation -- Windows NFS-mounted shares
Get-CimInstance -ClassName Win32_MappedLogicalDisk | Where-Object {$_.ProviderName -like '*nfs*'}
Or via PowerShell NFS module (if loaded):
Get-NfsMountedClient
```

#### Mount NFS Share

```
bash on appliance
mount -t nfs <server>:/path /mount/point
```

#### PowerShell equivalent:

```
PowerShell on workstation -- requires Windows NFS client feature
mount \\<server>\<path> Z:
Or with named-mount cmdlet (from NFS module):
New-PSDrive -Name Z -PSProvider FileSystem -Root '\\<server>\<path>' -Persist
```

---

## 21.4 Time Synchronization

### *ntpd*

**Description:** Forces immediate time sync with NTP server.

#### Syntax:

```
bash on appliance (root)
ntpd -s <ntp-server>
```

#### PowerShell equivalent:

```
PowerShell on workstation -- Admin elevation required
w32tm /config /manualpeerlist:"<ntp-server>" /syncfromflags:manual /update
Restart-Service w32time
w32tm /resync /force
```

---

`timedatectl`

**Description:** Shows comprehensive time configuration.

**Syntax:**

```
bash on appliance
timedatectl
```

**PowerShell equivalent:**

```
PowerShell on workstation
w32tm /query /status
w32tm /query /configuration
Get-TimeZone
Get-Date
```

---

## 22. Log Analysis

### 22.1 Real-time Monitoring

`tail -f`

**Description:** Follow log file in real-time.

**Examples:**

```
bash on SDDC Manager / vCenter via SSH (paths are appliance-side)
Domain Manager log
tail -f /var/log/vmware/vcf/domainmanager/domainmanager.log

Lifecycle Manager log
tail -f /var/log/vmware/vcf/lcm/lcm.log

Operations Manager log
tail -f /var/log/vmware/vcf/operationsmanager/operationsmanager.log

vCenter vpxd log
tail -f /var/log/vmware/vpxd/vpxd.log
```

*Appliance-only — no PowerShell sibling. The log paths above are inside the SDDC Manager and vCenter appliances; SSH in and tail them there. To stream a remote log to a Windows workstation, pipe over SSH: `ssh root@<sddc-manager> 'tail -f /var/Log/vmware/vcf/domainmanager/domainmanager.Log'` (works from PowerShell with OpenSSH client installed).*

---

`tail -n`

**Description:** View last N lines.

**Syntax:**

```
bash on SDDC Manager via SSH
```

```
tail -100 /var/log/vmware/vcf/domainmanager/domainmanager.log
```

*Appliance-only — no PowerShell sibling.*

---

## 22.2 Log Searching

### **grep** for errors

```
bash on SDDC Manager via SSH
grep -i error /var/log/vmware/vcf/domainmanager/domainmanager.log
```

*Appliance-only — no PowerShell sibling.*

### **grep** for exceptions

```
bash on SDDC Manager via SSH
grep -i exception /var/log/vmware/vcf/lcm/lcm.log
```

*Appliance-only — no PowerShell sibling.*

### **Filter by date**

```
bash on SDDC Manager via SSH
grep "2026-02-04" /var/log/vmware/vcf/domainmanager/domainmanager.log
```

*Appliance-only — no PowerShell sibling.*

### **Search compressed logs**

```
bash on SDDC Manager via SSH
zgrep -i error /var/log/vmware/vcf/domainmanager/domainmanager.log.gz
```

*Appliance-only — no PowerShell sibling.*

---

## 22.3 journalctl Commands

### **View service logs**

```
bash on SDDC Manager via SSH
journalctl -u vcf-services
```

*Appliance-only — no PowerShell sibling.*

### **Follow in real-time**

```
bash on SDDC Manager via SSH
journalctl -u vcf-services -f
```

*Appliance-only — no PowerShell sibling.*



### Time-based filter

```
bash on SDDC Manager via SSH
journalctl -u vcf-services --since "1 hour ago"
```

*Appliance-only — no PowerShell sibling.*

### Error-level only

```
bash on SDDC Manager via SSH
journalctl -u vcf-services -p err
```

*Appliance-only — no PowerShell sibling.*

## APPENDICES

### Appendix A: Quick Reference Cards

#### ESXi Quick Commands

Task	Command
Set FQDN	<code>esxcli system hostname set --fqdn=host.domain</code>
Regenerate certs	<code>/sbin/generate-certificates</code>
Restart services	<code>services.sh restart</code>
Check SSD status	<code>esxcli storage core device list \   grep 'Is SSD'</code>
Clear partitions	<code>partedUtil mklabel /dev/disks/[device] gpt</code>
Disable vSAN auto	<code>esxcli vsan storage automode set --enabled=false</code>
Leave vSAN	<code>esxcli vsan cluster leave</code>
List vmkernel NICs	<code>esxcfg-vmknic -l</code>
MTU test	<code>vmkping -d -s 1572 &lt;ip&gt;</code>
NSX proxy status	<code>/etc/init.d/nsx-proxy status</code>
Restart hostd	<code>/etc/init.d/hostd restart</code>
Restart vpxa	<code>/etc/init.d/vpxa restart</code>

#### SDDC Manager Quick Commands

Task	Command
Check VCF services	<code>systemctl status vcf-services</code>

Task	Command
Restart VCF services	<code>systemctl restart vcf-services</code>
Collect log bundle	<code>/opt/vmware/sddc-support/sos --log-bundle</code>
Health check	<code>/opt/vmware/sddc-support/sos --health-check</code>
View Domain Manager log	<code>tail -f /var/log/vmware/vcf/domainmanager/domainmanager.log</code>
Check disk space	<code>df -h</code>
Clean /tmp	<code>rm -rf /tmp/*</code>

## vCenter Quick Commands

Task	Command
Check all services	<code>service-control --status --all</code>
Restart all services	<code>service-control --restart --all</code>
Restart vpxd only	<code>service-control --restart vpxd</code>
View vpxd log	<code>tail -f /var/log/vmware/vpxd/vpxd.log</code>

## vSAN Quick Commands

Task	Command
Cluster info	<code>esxcli vsan cluster get</code>
Health checks	<code>esxcli vsan health cluster list</code>
Storage list	<code>esxcli vsan storage list</code>
Network list	<code>esxcli vsan network list</code>
Resync status	<code>esxcli vsan debug resync summary</code>

## NSX Quick Commands

Task	Command
Transport nodes	<code>get transport-nodes</code>
Cluster status	<code>get cluster status</code>
Firewall rules	<code>get firewall rules</code>
Logical switches	<code>get logical-switches</code>

## Appendix B: Common Ports

Service	Port	Protocol	Description
SSH	22	TCP	Secure Shell
DNS	53	TCP/UDP	Domain Name Service
HTTP	80	TCP	Web traffic
NTP	123	UDP	Time synchronization
HTTPS	443	TCP	Secure web traffic
ESXi Web	443	TCP	ESXi Host Client
vCenter	443	TCP	vSphere Client
NSX Manager	443	TCP	NSX UI
VMware Auth	902	TCP	vSphere authentication
NSX Messaging	1234	TCP	NSX Manager communication
NFS	2049	TCP	Network File System
Portmapper	111	TCP	NFS portmapper
vMotion	8000	TCP	vMotion traffic
Custom HTTPS	8443	TCP	Offline depot
vSAN Clustering	12345-23451	UDP	vSAN traffic

## Appendix C: Log File Locations

### SDDC Manager Logs

Log Path	Description
/var/log/vmware/vcf/domainmanager/domainmanager.log	Domain Manager main log
/var/log/vmware/vcf/domainmanager/domainmanager-gc.log	Domain Manager GC log
/var/log/vmware/vcf/lcm/lcm.log	Lifecycle Manager log
/var/log/vmware/vcf/lcm/upgrade/	Upgrade-specific logs
/var/log/vmware/vcf/operationsmanager/operationsmanager.log	Operations Manager log
/var/log/vmware/vcf/sos/sos.log	SoS utility log
/var/log/vmware/vcf/commonsvcs/commonsvcs.log	Common services log
/var/log/vmware/vcf/sddc-support/sddc-support.log	Support bundle log
/var/log/nginx/error.log	NGINX error log
/var/log/postgresql/postgresql-*.log	PostgreSQL logs

### vCenter Logs

Log Path	Description
/var/log/vmware/vpxd/vpxd.log	Main vCenter Server log
/var/log/vmware/vsphere-client/logs/vsphere_client_virgo.log	vSphere Client log
/var/log/vmware/vpostgres/postgresql*.log	PostgreSQL database logs
/var/log/vmware/sso/vmware-sts-idmd.log	SSO/Lookup service log

## ESXi Logs

Log Path	Description
/var/log/vmkernel.log	VMkernel log
/var/log/hostd.log	Host daemon log
/var/log/vpxa.log	vCenter agent log
/var/log/nsx-syslog.log	NSX agent log

## Appendix D: Certificate Locations

Component	Certificate Path
ESXi Certificate	/etc/vmware/ssl/rui.crt
ESXi Private Key	/etc/vmware/ssl/rui.key
vCenter	/etc/vmware-vpx/ssl/
SDDC Manager	/etc/vmware/vcf/commonsvcs/
NSX Manager	/config/cluster-manager/

## Appendix E: Default Credentials

Component	Username	Default Password
Cloud Builder SSH	root	vmware
Java Keystore	-	changeit
ESXi Hosts	root	(set during install)
vCenter	administrator@vsphere.local	(set during install)

## Appendix F: VMware Workstation VMX Settings

For nested lab environments, mark virtual disks as SSDs:

```
sata0:0.virtualSSD = 1 # SATA controller 0, disk 0
sata0:3.virtualSSD = 1 # SATA controller 0, disk 3
scsi0:0.virtualSSD = 1 # SCSI controller 0, disk 0
nvme0:0.virtualSSD = 1 # NVMe controller 0, disk 0
```

**Format:** [controller]:[disk].virtualSSD = 1

## Appendix G: VCF 9.0 Upgrade Order

1. **SDDC Manager** - First (orchestrates all upgrades)
2. **vCenter Server** - Management plane
3. **NSX Manager** - Networking control plane
4. **ESXi Hosts** - Compute (rolling upgrade)
5. **vSAN** - Storage
6. **VCF Operations** - Monitoring (if deployed)

## Document Information

Field	Value
<b>Document Title</b>	VCF 9.0 Unified Command Reference
<b>Version</b>	4.1 (Option B PowerShell-sibling pass)
<b>Created</b>	February 2026
<b>Last Updated</b>	May 2026
<b>Total Commands</b>	280+
<b>Bash blocks</b>	168
<b>PowerShell siblings</b>	28 (workstation-runnable: openssl cert generation/viewing/verify/s_client, SDDC Manager REST API curl, generic curl downloads/headers, nc / Test-NetConnection , nslookup / Resolve-DnsName , /etc/resolv.conf / Get-DnsClientServerAddress , NFS show/list/mount, ntpdate / w32tm , timedatectl / w32tm )
<b>Appliance/ESXi-only blocks</b>	140 (esxcli, vim-cmd, vmkfstools, vsish, vmkping, partedUtil, esxcfg-, esxstop, vscsiStats, /etc/init.d/, services.sh, generate-certificates, service-control, vamicli, systemctl, journalctl, sos, psql/pg_dump, keytool, log file paths in /var/log/)
<b>Categories</b>	20
<b>Source Files</b>	VCF9-Ultimate-Command-Reference.md, VCF9_Combined_Command_Reference.txt
<b>Translation Reference</b>	<a href="#">../POWERSHELL-TRANSLATION-REFERENCE.md</a>